

ML Major 3

Submitters:

Husain Azaqy: 213382104

Galor Lazar: 325028876

(Q1) In your report, derive (in English) the analytical partial derivative $\frac{\partial}{\partial b} L(\underline{w}, b) \in \mathbb{R}$.

$$\begin{aligned}\frac{\partial}{\partial b} L(w, b) &= \frac{\partial}{\partial b} \left(\frac{1}{m} \sum_{i=1}^m (w^T \cdot x_i + b - y_i)^2 \right) = \frac{1}{m} \sum_{i=1}^m 2 \cdot (w^T \cdot x_i + b - y_i) \\ &= 2b + \frac{2}{m} \sum_{i=1}^m (w^T \cdot x_i - y_i)\end{aligned}$$

(Q2) Using your preprocessed (and normalized) dataset and `contamination_level` as our target, generate a plot that compares the numerical gradients to the analytical ones.

Do this by running the following command:

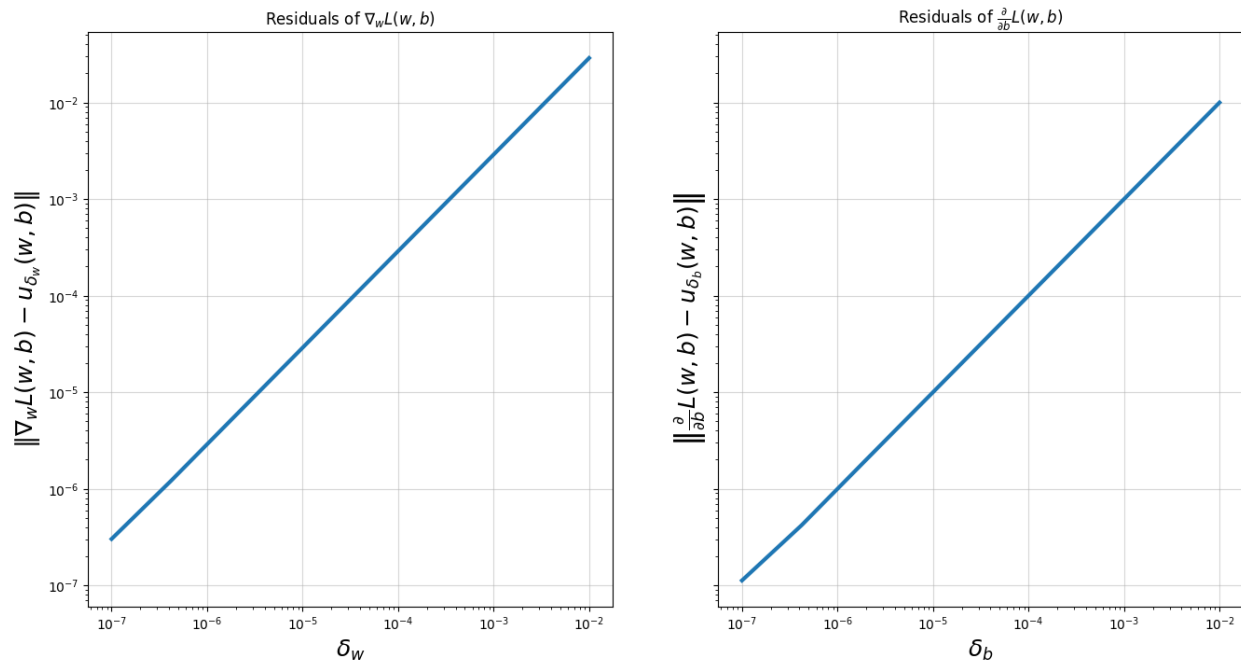
```
compare_gradients(X_train, y_train, deltas=np.logspace(-7, -2, 9))
```

Important: `X_train` should hold the features of your **normalized** training subset.

`y_train` should hold the `contamination_level` subset training labels.

Attach the plots to your report. No need to discuss these plots, but make sure that they make sense.

Residuals of analytical and numerical gradients



(Q3) We now want to evaluate the effects of different learning rates on the learning procedure. Run the following command that plots a graph of the training and validation losses as a function of the iteration number for different learning rates.

```
test_lr(X_train, y_train, X_val, y_val)
```

Important: `X_val` should hold the features of your (preprocessed) validation subset.

`y_val` should hold the `contamination_level` subset validation labels.

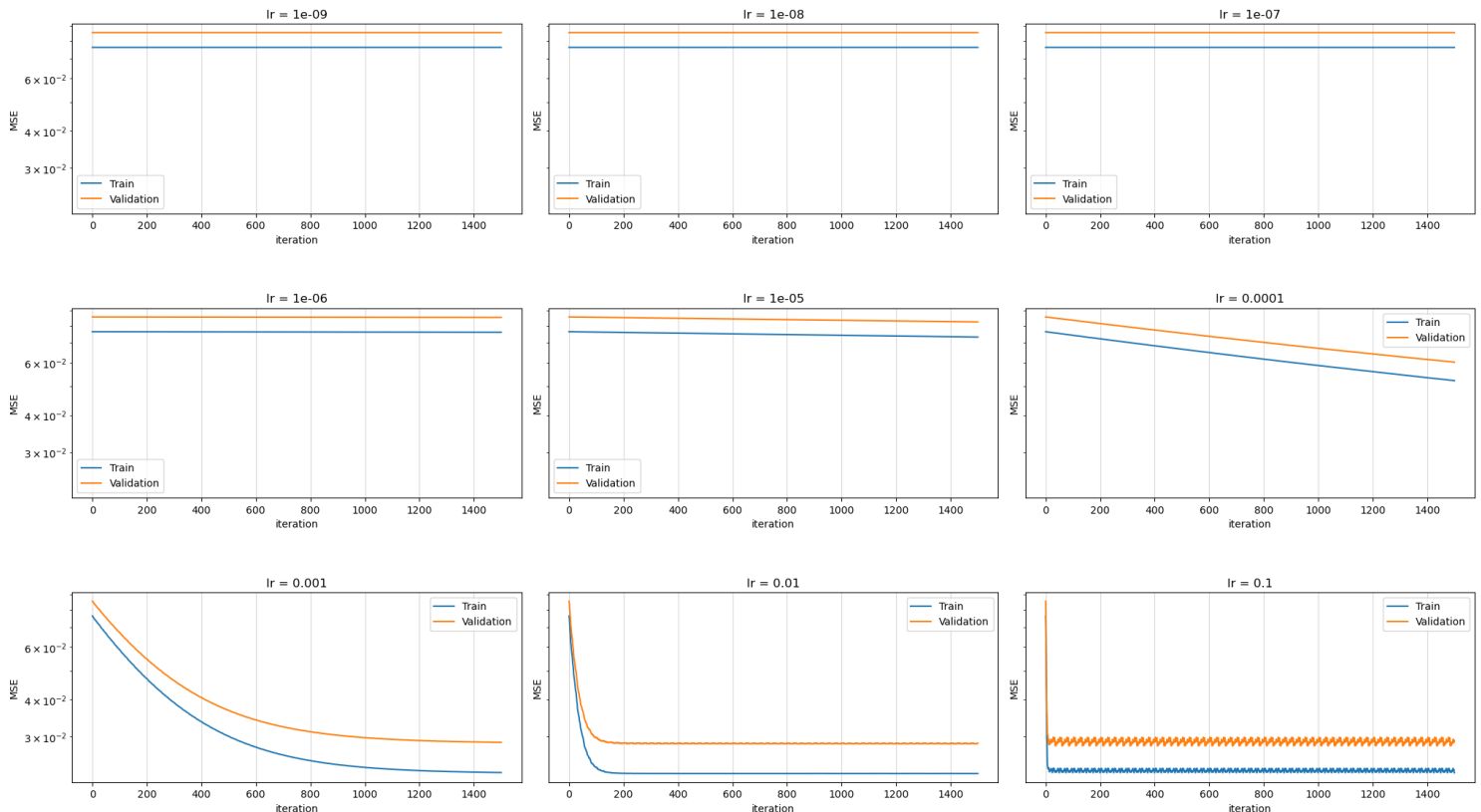
Note: If your model did not converge with any learning rate, you are allowed to alter the `lr_list` variable (but explain this in your report).

This part should also help you verify your implementation (loss should decay).

Attach the plots to your report, briefly discuss the results and justify the demonstrated behaviors. Which learning rate is “optimal” (briefly explain)?

For the best learning rate, does it make sense to increase the number of gradient steps (instead of the default 1500 steps)? Explain.

Convergence per learning rate



```

lr size = 1e-09, Best train loss = 0.07631351895494184, Best validation loss = 0.08555616713470707
lr size = 1e-08, Best train loss = 0.07631064407314496, Best validation loss = 0.08555315558374867
lr size = 1e-07, Best train loss = 0.07628190377997457, Best validation loss = 0.08552304891417745
lr size = 1e-06, Best train loss = 0.0759953516165901, Best validation loss = 0.08522286446002557
lr size = 1e-05, Best train loss = 0.07321322101883175, Best validation loss = 0.08230751059298688
lr size = 0.0001, Best train loss = 0.05227381597739127, Best validation loss = 0.060302988271156405
lr size = 0.001, Best train loss = 0.022724319344086995, Best validation loss = 0.02869937498193159
lr size = 0.01, Best train loss = 0.022548071172539496, Best validation loss = 0.0283671790405016
lr size = 0.1, Best train loss = 0.02268955267400382, Best validation loss = 0.027956543466381024

```

We can see that in cases where the $lr < 0.0001$ the learning is very slow and doesn't converge. That happens because in these cases the lr in every iteration is too small, so it doesn't have a significant enough effect on the model accuracy.

So, if we take larger learning rates, we will get better accuracies and better convergence as well as we doing more iterations.

As we can see, the best performance we get is on $lr=0.1$. Also, in its graph, we can see that it converges even before the first 200 iterations. After those iterations, there are only small "zigzags" that are in almost consistent cycles that don't get any better than what we got after the first 200 iterations.

By this, we know we can get the best accuracy of the $lr=0.1$ in a much smaller amount of iterations than 1500 steps.

If so, there is no sense to do a higher amount of 1500 iterations.

(Q4) Create a [DummyRegressor](#). Evaluate its performance using `cross-validation`.

In your report, fill in the cross-validated errors of the regressor.

Model	Section	Train MSE	Valid MSE
		Cross validated	
Dummy	2		

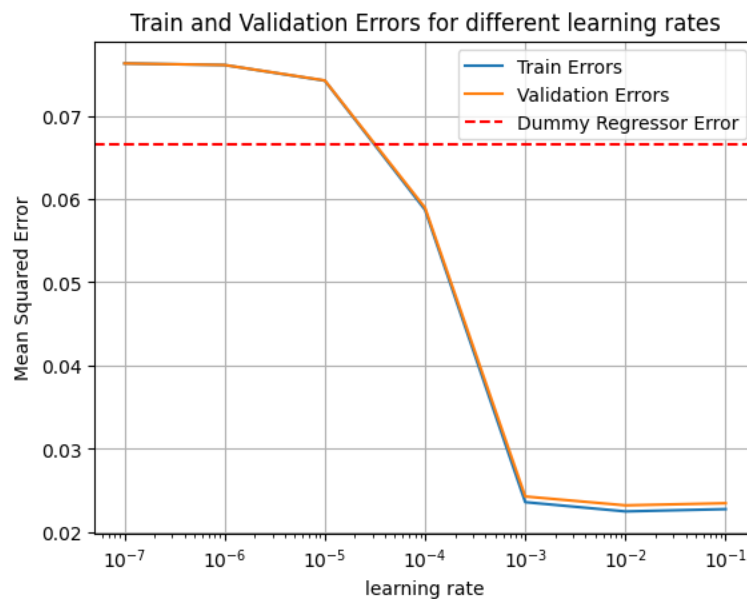
Model	Section	Train MSE	Valid MSE
		Cross validation	
Dummy	2	0.06624823283856855	0.06655387736460414

(Q5) Create an instance of your custom LinearRegressor and evaluate its performance using cross-validation, **remember to tune the LR!**

Report the appropriate plots and values detailed above in “the repeated tuning process”.

Fill in the cross-validated errors of the regressor yielded by the optimal hyperparameter.

Model	Section	Train MSE	Valid MSE
		Cross validated	
Dummy	2	filled	filled
Linear	2		



Optimal learning rate: 0.01
 Training error at optimal learning rate: 0.02247694080209269
 Validation error at optimal learning rate: 0.023206892546040265

Model	Section	Train MSE	Valid MSE
		Cross validation	
Dummy	2	0.06624823283856855	0.06655387736460414
Linear	2	0.02247694080209269	0.023206892546040265

(Q6) Had we chosen not to normalize features beforehand, would the training performance of these two models have changed (assume there are no numerical errors)? Explain.

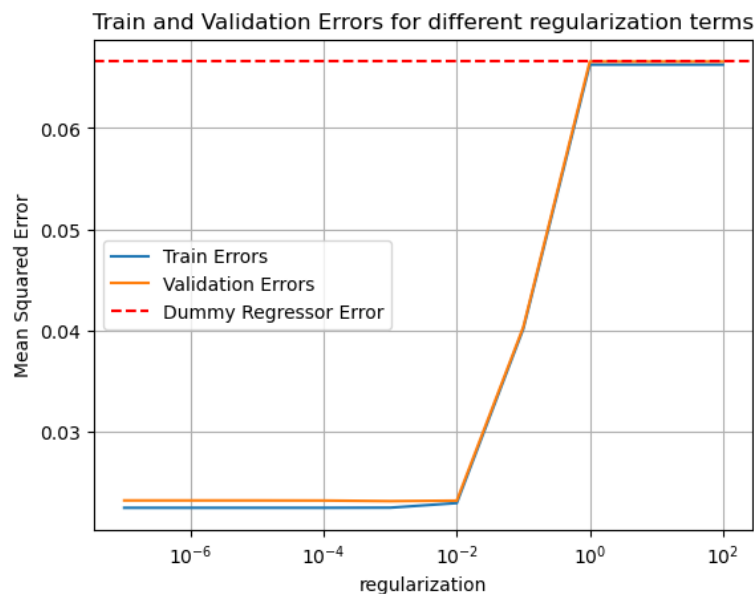
The Dummy model always predicts the average 'contamination_level' of the training set. This model doesn't take into account the features, it only takes the y' value and returns the average. If so, normalizing the features wouldn't affect the model predictions and performance.

For the Linear model, the model may converge slower with no normalization, and the learned coefficients may be less meaningful or skewed because of the influence of larger in scale features. So it can hurt the model's performance and predictions of new data.

All in all, not normalizing features won't change the Dummy model performance but may hurt the Linear model performance.

(Q7) Tune the regularization strength of the regressor. Follow the repeated tuning process described earlier. Remember to attach the required plot and specify the optimal strength with its validation error.

Remember that plot should have suitable titles, axis labels, grid lines, etc.



Optimal regularization alpha: 0.001

Validation error at optimal regularization alpha: 0.02313150258151016

Training error at optimal regularization alpha: 0.022487430212685115

(Q8) Fill in the cross-validated errors of the regressor yielded by the optimal hyperparameter.

Model	Section	Train MSE	Valid MSE
		Cross validated	
Dummy	2	filled	filled
Linear	2	filled	filled
Lasso Linear	3		

Model	Section	Train MSE	Valid MSE
		Cross validation	
Dummy	2	0.06624823283856855	0.06655387736460414
Linear	2	0.02247694080209269	0.023206892546040265
Lasso Linear	3	0.022487430212685115	0.02313150258151016

(Q9) Specify the 5 features having the 5 largest coefficients (in absolute value) in the resulting regressor, from the largest to the smallest (among these 5).

Feature: PCR_04, Coefficient: 0.19188858316551435

Feature: PCR_02, Coefficient: 0.072140668782269

Feature: PCR_06, Coefficient: 0.04064930398881352

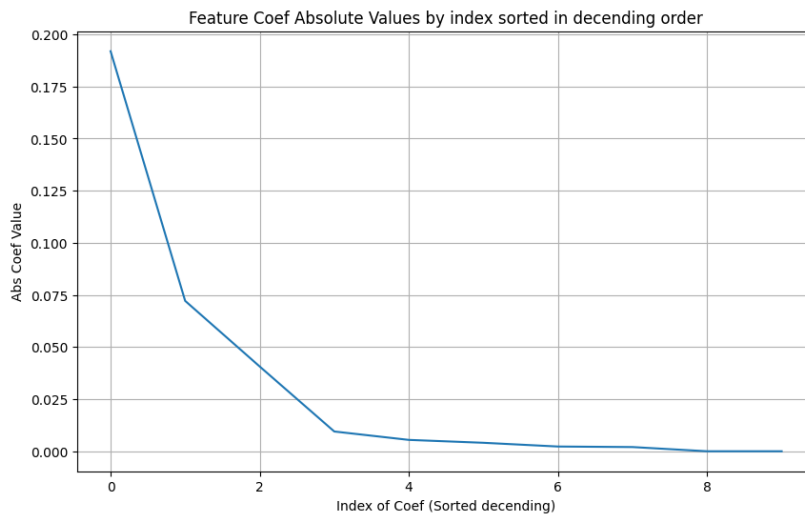
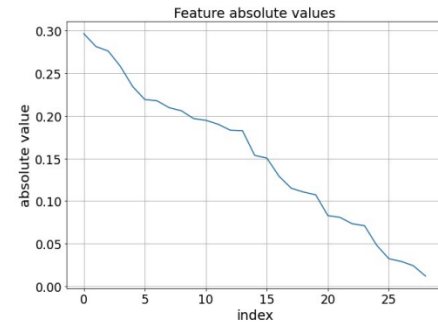
Feature: PCR_10, Coefficient: 0.00947751005415817

Feature: PCR_09, Coefficient: 0.00544732875497806

We now wish to visualize the resulting coefficients.

(Q10) Sort and plot the absolute values of the learned coefficients. The x-axis should be the index of the parameter from largest to smallest.

The result should roughly look like the illustration to the right (in terms of axis and structure, the values her are made up).



(Q11) Why is the magnitude of the coefficients interesting? Explain briefly.

The magnitude of the coefficients is interesting because it indicates the effect of a feature on the final target variable.

Let us remember that the features are normalized. As much as the coefficient of a specific feature is larger (in absolute value) it means that this feature has a larger effect on the target variable. In the same way, a smaller coefficient indicates a weaker influence and correlation.

By checking which feature coefficients are the largest, we can analyze the magnitude of the coefficients and identify which features are most important for the model's predictions.

Knowing which features have the largest coefficients allows us to focus on the most relevant features. This can be very useful in high-dimensional datasets where many features might be irrelevant.

(Q12) Had we chosen not to normalize features beforehand, would the training performance of the Lasso model have changed (assume there are no numerical errors)? Explain.

If the features are not normalized, they may have different scales.

With non-normalized features, since λ is applied equally to all coefficients, the scale of features affects the Lasso l1 regularization.

Larger scaled, less important features might not have their coefficients reduced to zero due to their scale, while more important small scale features might be shrunk with zero coefficients.

This may happen because Lasso “variable selection” can be affected by feature scale. As a result, features with larger scales might be under penalized, and those with smaller scales might be over-penalized. This imbalance can lead to retaining less relevant features and discarding important ones, ultimately skewing the model’s results and hurting its performance.

(Q13) How would you expect the coefficients of the trained model to change, should we have used the Ridge regressor instead? Explain.

Regarding the trained lasso model coefficients, like before we expect for large scaled features to have non zero coefficients, and for smaller scaled features to have closer to zero coefficients.

Using the ridge regressor, since it has l2 regularization it tends to shrink coefficients to zero less than the l1 regularization used in the lasso model.

The ridge model is less sensitive to the effects of not normalizing features since it tends to shrink coefficients rather than zeroing them out. This makes it more stable when features are not normalized compared to the lasso model.

(Q14) Why is re-normalization important after applying a polynomial mapping?

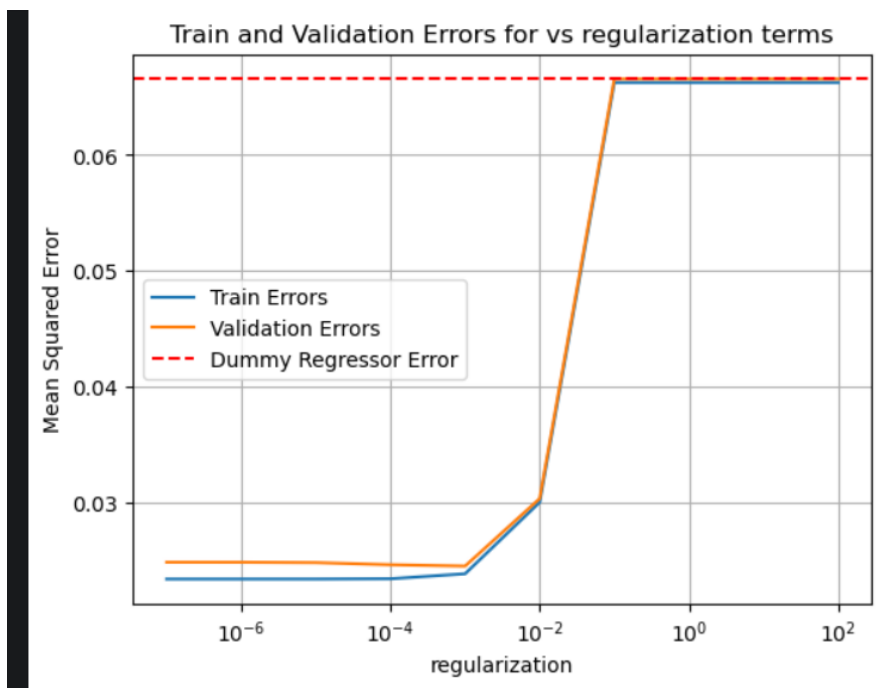
Hint: Your answer should revolve around the degrees of the different monomials and focus on mathematical/"optimizational" considerations rather than on statistical ones like in Lecture 08.

The magnitude of a polynomial feature involves multiple monomials and powers of them, potentially changing the scale of the new polynomial features relative to the original base features.

We already saw in the lectures that the magnitude of the features can effect learning especially when dealing with gradient descent.

(Q15) Tune the regularization strength of a Lasso regressor with the polynomial mapping using cross validation ($k = 5$ as always). (Remember: on Page 12 in HW2 you saw how to set hyperparameters inside a pipeline. Use [cross_validate](#).)

Remember to attach required plots, optimal strengths, and optimal validation errors.



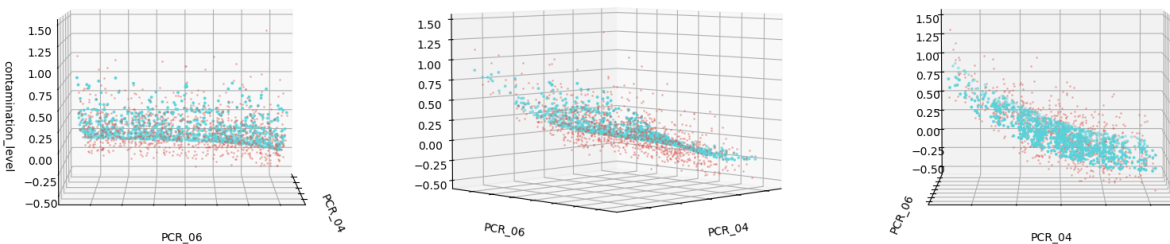
Optimal regularization alpha: 0.001

Validation error at optimal regularization alpha: 0.024509419331913352

Training error at optimal regularization alpha: 0.02384382946755397

(Q16) We will now visualize the polynomial model you just trained. Use the `new plot3d` function to plot all the training samples and their true labels (as before). Pass your model predictions in a list to the `predictions` keyword to compare the true labels to the predicted values. Attach the plot to your report.

contamination_level spread relative to PCR_04 & PCR_06

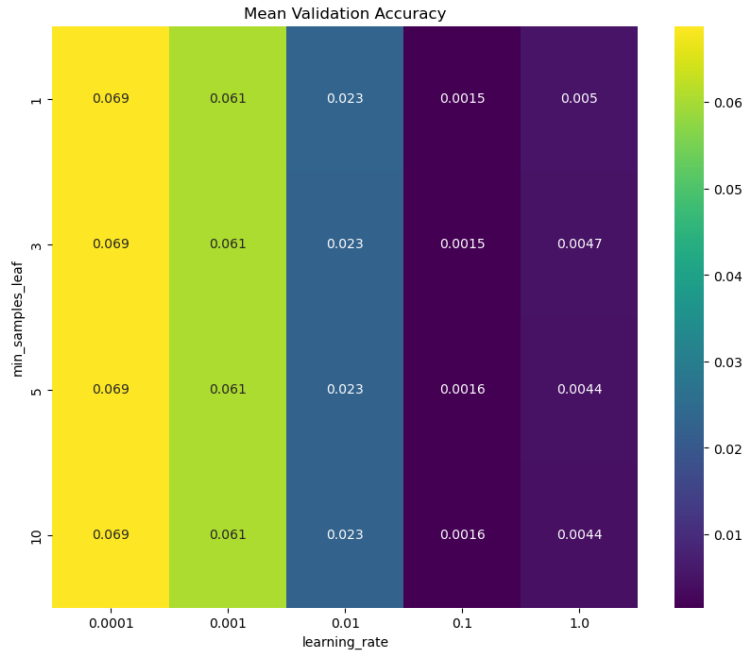


(Q17) Fill in the train and validation errors of the regressor yielded by the best performing hyperparameter. Remember to compute the errors using cross-validation.

Model	Section	Train MSE	Valid MSE
		Cross validated	
⋮	⋮	filled	filled
Polynomial Lasso	4		

Model	Section	Train MSE	Valid MSE
		Cross validation	
Dummy	2	0.06624823283856855	0.06655387736460414
Linear	2	0.02247694080209269	0.023206892546040265
Lasso Linear	3	0.022487430212685115	0.02313150258151016
Polynomial Lasso	4	0.02384382946755397	0.024509419331913352

(Q18) Tune the 'learning_rate', and 'min_samples_leaf' parameters of the [GradientBoostingRegressor](#) model using cross-validation with a grid search (see Q8 in Major HW2). For the 'loss' parameter, use 'huber' (for further details regarding the Huber loss see [here](#)). Use a similar pipeline to the one created earlier in this section. Remember to attach required heatmaps, optimal hyper-parameters, and optimal train and validation errors (use the default values for all other parameters).



```
Optimal Hyper params: {'learning_rate': 0.1, 'min_samples_leaf': 3}
Optimal Train Error: 0.0015133026206487735
Validation Error: 0.0010523804993928887
```

(Q19) Fill in the train and validation errors of the regressor yielded by the best performing hyperparameter. Remember to compute the errors using cross-validation.

Model	Section	Train MSE	Valid MSE
		Cross validated	
⋮	⋮	filled	filled
GBM Regressor	5		

Model	Section	Train MSE	Valid MSE
		Cross validation	
Dummy	2	0.06624823283856855	0.06655387736460414
Linear	2	0.02247694080209269	0.023206892546040265
Lasso Linear	3	0.022487430212685115	0.02313150258151016
Polynomial Lasso	4	0.02384382946755397	0.024509419331913352
GBM Regressor	5	0.0015133026206487735	0.0010523804993928887

(Q20) Complete the entire table

Model	Section	Train MSE	Valid MSE	Test MSE
		Cross validated		Retrained
Dummy	2	filled	filled	
Linear	2	filled	filled	
Linear Lasso	3	filled	filled	
Polynomial Lasso	4	filled	filled	
GBM	5	filled	filled	

Which model performed best on the test set?

Briefly discuss the results in the table (from an overfitting and underfitting perspective, or any other insightful perspective).

Model	Section	Train MSE	Valid MSE	Test MSE
		Cross validation		Retrained
Dummy	2	0.06624823283856855	0.06655387736460414	0.07060
Linear	2	0.02247694080209269	0.023206892546040265	0.02575
Lasso Linear	3	0.022487430212685115	0.02313150258151016	0.02561
Polynomial Lasso	4	0.02384382946755397	0.024509419331913352	0.02779
GBM Regressor	5	0.0015133026206487735	0.0010523804993928887	0.00099

- The model that performed best on the test set is the GBM Regressor model with loss of 0.00099.
- The Dummy model has the worst performance of all the 5 models. It underfits the data, because it is too simple to capture the underlying data patterns.
- Both Lasso Linear and Linear models show similar performance in training and validation. They don't seem to have directed signs of overfitting or underfitting. Since training and validation accuracies are similar and not bad.
- The Polynomial Regressor model has lower Train and Valid MSE, than the 2 previous models. However, the 'Test MSE' is higher than the Linear and Lasso models, so it can mean that the model may be overfitting the training data.
- The GBM Regressor is the best of the 5 models, in the Train, Valid and Test MSE's. It means that this model is good and it success to generalizes very well to new data, by capturing the patterns of the training data. That is since all train ,validation and test accuracies are good and similar.
- See attached Histogram below:

